# Convoy: shipment tracking prototype

## The cut I chose and why

I built two roles and only the screens that answer one question each: for the customer, "is my order on time, and when does it arrive?"; for ops, "which shipments do I chase first?".

- Customer: shipment list, detail page, notification feed.
- Ops: Today, exceptions queue, portfolio table, detail page.
- Ops is scoped to two of the four sites, matched on origin or destination.
- One shipment sits outside that scope on purpose: its URL says "not found" and search matches nothing.

Left out, and why:

- A map. A good one costs hours, a cheap one looks generic. The leg-by-leg timeline carries the same information.
- Real-time operator integrations. The brief is frontend only; one scripted live event shows how freshness would feel.
- Document upload, analytics, dark mode, multiple languages. None changes the answer to the two questions.
- Free-form LLM search. It needs a network and a key; a canned parser is honest about what is simulated.
- Automated tests. I spent the hours on the product and checked by hand: lint, build, both roles at 1440 and 360 px, keyboard, reduced motion. A real gap.

## Where and how the AI adds value

Six features, each on the screen where someone needs it, all simulated with deterministic rules.

- Every AI output carries a small "Convoy" stamp and a "Why" line naming the facts it used.

| Feature | Where | Based on |
|---|---|---|
| Exception management | Ops Today, exceptions queue | Delay, customs hold, silence, missing documents, port incidents. Each row proposes one next action. |
| Arrival estimates | Shipment detail, both roles | Confirmed delays plus typical customs time, with a confidence word and the reason. |
| Consolidation | Timeline, ops detail | Six operator vocabularies ("Gate-in", "VSL DEP", "Dédouané") mapped to one set of milestones, raw feed beside it. |
| Daily summary | Ops Today | Live counts and ranked next actions, recomputed after every ops action. |
| Natural-language search | Ops portfolio | "shipments to France this week running late" becomes three chips and a filtered table. Unused words are marked ignored. |
| Proactive notices | Exceptions queue, then the customer's banner and feed | Order, route, estimate, confidence, cause. Ops edits the draft before sending. |

Every action says who receives it and how:

- Each dialog shows recipient and channel ("Baltic Customs · email"). Notices go to the customer's Convoy inbox and email. The activity log records the channel.
- Documents follow ownership. The manufacturer issues the invoice and packing list, so ops sends them to the broker or carrier. The carrier issues the bill of lading or CMR and the broker the customs declaration, so those are requested. The customer never gets a document request.
- Notices are capped at 600 characters; long ones are clamped to three lines in the customer view until opened.
- Toasts link to the shipment they are about.

A prediction never passes as a fact:

- A timestamped operator event is confirmed and drawn solid. Anything planned or estimated is dashed or hatched.
- One `Estimate` component is the only code allowed to render an estimate.
- If nothing moved the date, the interface says "Planned", not "Estimated".
- An unknown operator status shows as "Not mapped" and stays raw. A quiet feed shows the customer "Awaiting update". I would rather show a gap than guess.

## Technical approach

Store facts, derive judgments.

- React 19, Vite, react-router for real URLs, plain CSS with custom-property tokens, Barlow, lucide icons. No backend, UI kit or state library: nothing to fight.
- The store (a context with a reducer) holds operator events, the plan, documents and what ops did.
- Risk, estimates, summary, search and drafts are pure functions in `src/ai/`, run through selectors. Ops actions reach the customer view with no extra wiring, and that folder is what production replaces.
- Risk weights are plain numbers: +4 for a customs hold, +3 for 24 hours of silence, +2 per day of slip, capped at +6. Same inputs, same ranking.
- 24 shipments, six operators, a fixed "now" of 13 Oct 2026, 14:30. The demo renders the same on any machine.

Production path for the AI:

- Operator statuses: an LLM maps free text to milestones, validated against a schema. Rules stay as fallback; low confidence lands in "Not mapped".
- Arrival estimates: a model on lane history plus vessel and port feeds, with thresholds deciding between a date, a range or "no update yet".
- Notices: the model drafts, a human approves every message to a customer.
- Search: the LLM returns the filter object the dropdowns use, so chips stay inspectable and a bad parse can be corrected.
- Evaluation: a labelled set of real statuses and past delays scores each change before release. Webhooks where operators offer them, polling elsewhere.

## Assumptions, limitations and next steps

Assumed: one customer and one ops persona are enough to show scoping; English, metric units and EUR; operators expose timestamped events; the demo runs locally, so no deployment or auth.

Limitations:

- Reload resets all data.
- Search understands a fixed set of phrases.
- Risk and estimates are rules, not models. They cannot learn.
- One scripted live event.
- No auth, no tests.

Next steps, in order:

1. Real events behind a thin API with webhooks.
2. An evaluation set for the status mapper, before any LLM touches customer-facing text.
3. Tests for the risk and search rules, the cheapest to cover.
4. Notification preferences, document upload and a map, once I know what customers ask for.

I used AI coding tools (Claude Code) throughout. I made the scope, product and visual decisions and reviewed what it produced.